

Kalpion — system architecture

IFQM employee ideation platform

Repository state: main @ 0761264, revised with identity model and SSO design

15 September 2026

- | | |
|------------------------------|--------------------------------|
| 1. Overview | 9. Identity data model |
| 2. Runtime topology | 10. Sign-in flows |
| 3. Repository layout | 11. Single sign-on integration |
| 4. Frontend | 12. A request, end to end |
| 5. Backend | 13. Background work |
| 6. Multi-tenancy | 14. External services |
| 7. Data model and migrations | 15. Build, test and deployment |
| 8. Identity and access | 16. Configuration |

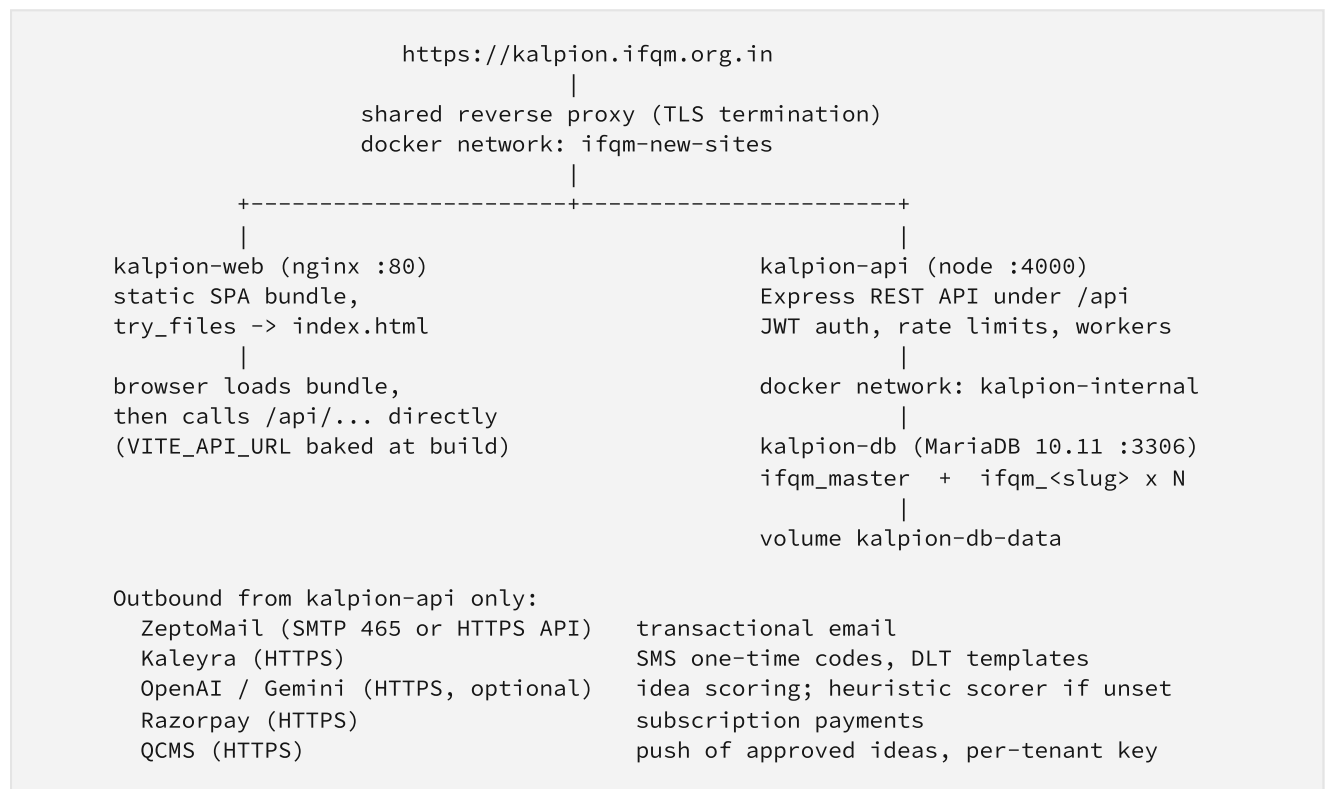
1 Overview

Kalpion is a multi-tenant web application in which employees of an organisation submit improvement ideas, the ideas are scored and routed up a configurable approval chain, and contributors are rewarded with points and leaderboards. Each organisation is a tenant with its own database schema, branding, users and settings. A separate platform-administrator role manages tenants, plans and billing without seeing tenant content.

It is a conventional three-tier system: a single-page React application talks over HTTPS to a Node/Express REST API, which reads and writes MariaDB. There are no message brokers, caches or other stateful services; timers inside the API process handle background work.

LAYER	TECHNOLOGY	WHERE IT RUNS IN PRODUCTION
Frontend	React 18, Vite, React Router, axios	Static files served by nginx (container <code>kalpion-web</code>)
API	Node 22, Express, mysql2 (raw SQL, no ORM)	Container <code>kalpion-api</code> , port 4000
Database	MariaDB 10.11	Container <code>kalpion-db</code> , named volume <code>kalpion-db-data</code>
Host	Docker Compose on a Linux VPS behind a shared reverse proxy	https://kalpion.ifqm.org.in

2 Runtime topology



The browser never talks to the database, and the API never serves the SPA. The web and API containers sit on the proxy's network so the proxy can reach both by container name; only the API is on the internal network with the database. The database publishes no host port.

3 Repository layout

```
ifqm/
  backend/
    server.js          entrypoint: config check, listen, start scheduler
    src/
      app.js           Express app factory (middleware pipeline)
      config/          env parsing, TLS rules for DB, SMS template registry
      routes/          one router per resource, mounted in routes/index.js
      controllers/     thin: parse request, call a service, respond
      services/        all business logic (ideas, auth, mail, SMS, billing...)
      database/        master pool, per-tenant pool cache, reconnect wrapper
      middleware/      auth + tenant attach, rate limiting, tenant quota, errors
      workers/         scheduler (timers) and bcrypt worker thread
      utils/           jwt, logger, respond, asyncHandler
    schema/tenant_schema.sql  template applied when a tenant is provisioned
    scripts/           setup, migrate, migrate-remote, export-remote, backup,
                        provision-tenant
    test/              HTTP invariant suite (drives the real API)
    Dockerfile
  frontend/
    src/
      App.jsx          route table
      pages/           one component per screen (~35)
      components/      shared UI, Layout/ holds AppShell, Sidebar, Topbar
      context/         Auth, Branding, Lang, Notif, Toast providers
      services/api.js   axios instance, token and org headers
      i18n/            en, hi, mr, kn, te, ta, ml
    nginx.conf, Dockerfile
  db/
    master.sql         the ifqm_master registry schema
    migrations/        forward-only SQL, 001..041
    docker-init/       GRANT applied on first database start
  docker-compose.yml
  .github/workflows/   ci.yml, deploy.yml
```

4 Frontend

A single Vite bundle. `App.jsx` declares every route; public routes (`/`, `/login`, `/signup`, `/reset-password`) are wrapped in `PublicRoute`, everything else in `PrivateRoute` inside `AppShell`, which supplies the sidebar, top bar and breadcrumbs. Tenant screens live at `/dashboard`, `/submit`, `/review`, `/all-ideas`, `/board`, `/leaderboard`, `/analytics`, `/admin` and so on; platform-administrator screens live under `/platform/...`.

Five React contexts carry cross-cutting state: `AuthContext` (session, role), `BrandingContext` (tenant logo and colour), `LangContext` (one of seven languages), `NotifContext` (unread notifications), `ToastContext`. Pages fetch through a single axios instance in `services/api.js`, whose request

interceptor attaches `Authorization: Bearer <jwt>` from `localStorage` along with the current organisation slug. The API base URL is a build-time constant (`VITE_API_URL`), so a bundle is tied to the deployment it was built for.

nginx serves the built files with `try_files $uri /index.html` so deep links resolve client-side. The frontend holds no secrets.

5 Backend

Request pipeline

`app.js` assembles middleware in this order; every request passes through it before reaching a route.

```
helmet (CSP default-src 'none', no referrer, HSTS when FORCE_HTTPS)
-> 308 redirect to https unless req.secure or x-forwarded-proto=https
-> CORS allow-list (CORS_ORIGIN, plus localhost in any port)
-> JSON / urlencoded body parsers, 1 MB limit
-> morgan access log
-> global rate limit: 300 requests / minute / IP
-> /api routes/index.js
    /auth /registrations /users /ideas /votes /comments /leaderboard
    /rewards /score /notifications /settings /challenges /categories
    /export /reports /upload /branding /support /integrations /platform
    /health
-> 404 handler
-> error handler (ApiError -> status + JSON; anything else -> 500, logged)
```

Layering

Each resource has a *route* file (URL, HTTP verb, guards), a *controller* (reads the request, calls one service, writes the response through `utils/respond.js`) and a *service* (all logic and SQL). Controllers do not contain SQL and services do not touch `req` or `res`; this is what lets the test suite and the scheduler call services directly. Guards are composed per route from `middleware/auth.js`: `optionalAuth`, `requireAuth`, `requireRole(...roles)` and `requirePlatformAuth`. Authentication endpoints additionally get `authLimiter` (30 requests per 15 minutes per IP).

Database access

`database/master.js` holds one pool to `ifqm_master`. `database/tenant.js` keeps an LRU cache of pools keyed by `host|db_name|user`, capped by `DB_MAX_POOLS`, and evicts the least recently used. Both are wrapped by `database/resilient.js`, which retries a query once on the connection-loss error codes a managed MySQL produces (`PROTOCOL_CONNECTION_LOST`, `ECONNRESET`, `ETIMEDOUT`, ...) and retires idle connections before the server's `wait_timeout` can. All queries are parameterised through `mysql2` prepared statements; every connection runs in UTC with `dateStrings` on, so the application never converts time zones.

6 Multi-tenancy

Isolation is at the schema level. `ifqm_master` is the registry; each organisation gets its own schema, `ifqm_<slug>`, created from `backend/schema/tenant_schema.sql` when a registration is approved. The application's database account holds a grant on the pattern `ifqm\_%`, which is what allows it to create a new schema at runtime without a server-wide privilege.

<code>ifqm_master</code> -----	<code>ifqm_vp</code> -----	<code>ifqm_jain</code> -----	<code>ifqm_acme</code> ... -----
<code>tenants (slug, db_name, db_host,</code> <code>domain, status, plan)</code>	<code>users</code>	<code>users</code>	<code>users</code>
<code>platform_admins</code>	<code>ideas</code>	<code>ideas</code>	<code>ideas</code>
<code>login_directory (identifier -> tenant)</code>	<code>idea_workflow</code>	<code>...</code>	<code>...</code>
<code>login_attempts, login_otps</code>	<code>org_settings</code>		
<code>tenant_registrations</code>	<code>email_queue</code>		
<code>plans, tenant_billing_events</code>	<code>notifications</code>		
<code>support_tickets(+messages)</code>	<code>challenges</code>		
<code>platform_settings, email_whitelist</code>	<code>leaderboard (view)</code>		
<code>sms_delivery_log, payment_attempts</code>			
<code>schema_migrations (ledger)</code>			

Resolution happens once per request in `attachTenantDb`: the JWT carries `org_slug`; `resolveTenantBySlug` looks the row up in `tenants` (checking status and non-payment holds); `getTenantPool` returns the cached pool; the result is placed on `req.tenant` and `req.db`. Everything downstream uses `req.db` and therefore cannot reach another tenant's schema.

Two details of the `tenants` row matter operationally. Credentials come from the environment (`APP_DB_USER` / `APP_DB_PASS`), never from the row; `db_user` and `db_pass` are stored empty. But the `host` does come from the row (`db_host` , falling back to `MASTER_DB_HOST`), so a registry copied from one server to another must have that column rewritten for the destination.

Platform administrators are a separate population in `ifqm_master.platform_admins`; they sign in with no organisation code, are guarded by `requirePlatformAuth`, and the `/platform` routes only ever aggregate counts across tenants.

7 Data model and migrations

The tenant schema (19 tables and one view) centres on `users` and `ideas`. An idea has one submitter, optional `idea_co_suggesters`, `idea_attachments`, `idea_reviewers` for committee review, an append-only `idea_workflow` log of every stage transition, and two vote tables (`idea_votes` for reviewers, `idea_community_votes` for everyone). `users.manager_id` forms the reporting line that sequential approval climbs. `org_settings` is a key/value table holding the approval chain, thresholds,

SLA days, email opt-out and similar per-tenant knobs. `email_queue` is per tenant so one organisation's backlog cannot delay another's.

Roles are an enum on `users.role`: trainee, employee, team_lead, project_lead, manager, senior_manager, executive, department_manager, plant_head, admin, super_admin. Points are stored on the user row (10 on submission, 25 on approval, 65 on implementation) and the `leaderboard` view aggregates them.

Migrations

Schema changes are SQL files under `db/migrations/`, numbered and forward-only. A file whose name ends in `_master.sql` targets the registry; any other file is applied to every tenant schema.

`ifqm_master.schema_migrations` records each (schema, file) pair that has run, so the runner only applies unrecorded pairs and is safe to re-run. Fixing a mistake means writing a new file. `npm run migrate` runs locally; `scripts/migrate-remote.mjs` runs against a remote server from an env file; the deploy workflow runs the migrator inside the API container after every release.

8 Identity and access

- **Identifiers.** A person signs in with a username, an email address or a mobile number. The organisation code is optional: `ifqm_master.login_directory` maps an identifier to its tenant, so the API can resolve the tenant before it knows which schema to query.
- **Passwords.** bcrypt, hashed in a worker-thread pool (`services/hashPool.js`) so a burst of sign-ins does not stall the event loop. A timing-safe dummy compare runs even when no user matches.
- **Lockout.** `login_attempts` in the registry counts failures per identifier; five failures lock for 15 minutes.
- **One-time codes.** `login_otps` issues six-digit codes by email, SMS or both for sign-in, password reset, organisation registration and mobile-number changes, with a resend throttle. Unknown identifiers get the same generic reply as known ones and no message is sent.
- **Sessions.** A signed JWT carries the user id, role and `org_slug`. On every authenticated request `loadLiveUser` re-reads the user row, so a role change, deactivation or password change takes effect immediately rather than at token expiry.
- **Platform admins.** Separate table, separate guard, and a verified second channel (phone) required for sensitive actions.
- **Transport.** Helmet headers, HSTS with preload when `FORCE_HTTPS` is on, CORS allow-list, global and per-route rate limits, a 1 MB body cap, uploads served only through an authenticated route scoped to the tenant.

9 Identity data model

Identity is split across the two schema types. The registry knows *who belongs where*; the tenant schema knows *who the person is*. Nothing in a tenant schema points back at the registry, and the registry never holds a foreign key into a tenant schema (MariaDB cannot enforce one across databases), so the links drawn with dashed lines below are maintained by application code, not by the database.

ifqm_master (registry) =====	ifqm_<slug> (one per organisation) =====
tenants ----- id PK <- - - - - - - - - - + slug UNIQUE domain (org web host) db_host, db_name status active suspended pending plan_id, billing_status, trial_ends_at last_login_at login_directory ----- identifier PK (email phone username) id_type tenant_id - - - - - - - - - - + tenant_slug user_id - - - - - - - - - - + ----- login_attempts ----- login_id PK (the identifier typed) attempts, locked_until ----- login_otps ----- identifier, id_type, purpose, channel code_hash, attempts, consumed_at, expires_at tenant_id, tenant_slug, user_id - - - - - + (copied at issue time, no FK) ----- platform_admins ----- id PK <-- support_tickets.assignee_id (FK) email UNIQUE, phone, password_hash email_verified_at, phone_verified_at ----- tenant_registrations ----- company_name, proposed_slug email_domain, contact_email, status (approved -> a tenants row + schema)	users ----- id PK <- - - - - - - - - - + employee_id UNIQUE (per tenant) username UNIQUE (platform-wide) email UNIQUE (per tenant) phone password_hash NOT NULL role ENUM (11 values) manager_id FK to users.id status active inactive password_changed_at must_change_password ----- referenced by (FK to users.id): ideas.submitter_id ideas.co_suggester_1_id / _2_id idea_reviewers.reviewer_id idea_workflow.actor_id idea_votes.user_id idea_community_votes.user_id idea_comments.user_id idea_co_suggesters.user_id notifications.user_id challenges.created_by password_reset_tokens.user_id user_import_jobs.actor_id ----- referenced by (FK to ideas.id): idea_reviewers, idea_attachments, idea_co_suggesters, idea_workflow, idea_votes, idea_community_votes, idea_comments ----- platform_login_activity ----- actor_type tenant_user platform_admin actor_id, actor_email, tenant_slug outcome success failure lockout ip, user_agent, time_zone, created_at

Read the diagram from the identifier inward. A sign-in string (email, phone or username) is the primary key of `login_directory`; that row names the tenant slug and the user id, which selects the schema and then the `users` row. Everything else in a tenant schema hangs off `users.id` through real foreign keys, so a user row is the root of that person's data in the organisation.

Uniqueness rules that matter for SSO

- `users.username` is unique across the whole platform, because it is a `login_directory` primary key. Two organisations cannot share a username.
- `users.email` is unique within a tenant. The same address can exist in two tenants, but only one of them can own it in `login_directory`; the other requires the organisation code at sign-in.
- `users.employee_id` is unique within a tenant and required.
- `users.password_hash` is `NOT NULL`. An account that never uses a password still needs a hash; the bulk importer sets a derived temporary one and flags `must_change_password`.
- `tenants.domain` is a web host used by `resolveTenant` when no slug is given (for example `vertexprecision.co.in`); `tenant_registrations.email_domain` is the mail domain the organisation registered with. Neither is currently used to authorise a sign-in.

How the directory is maintained

`services/directoryService.js` writes `login_directory` whenever a user is created, renamed or deactivated (`indexUser`, `claimUsername`, `releaseUsername`, `deindexUser`), and password login calls `indexUser` again after every success as a self-heal. If an identifier is missing from the directory, `resolveTenantByLogin` falls back to scanning every active tenant's `users` table, which is slower but keeps sign-in working. Any new way of creating users, including just-in-time provisioning from an identity provider, must call `indexUser` or the person will only be able to sign in with an organisation code.

10 Sign-in flows

Password

```
browser (LoginPage)      kalpion-api              ifqm_master              ifqm_
-----
POST /api/auth/login --> authLimiter (30 / 15 min / IP)
{email, password,        assertNotInMaintenance() -----> platform_settings
  org_slug?}             org_slug given?
                          yes: resolveTenant(slug, host) --> tenants
                          no : resolveTenantByLogin(id) --> login_directory
                          getTenantPool(tenant)
                          SELECT users WHERE email|phone|username ... AND status='active' -
                          bcrypt.compare (worker thread; dummy hash if no row)
                          fail: login_attempts += 1, lock at 5 --> login_attempts
                          ok  : clearFailedAttempts, indexUser --> login_directory
                              tenants.last_login_at = NOW() --> tenants
                              recordLogin(success) -----> platform_login_activity
                              signToken({user: session, org_slug, pwd_ts}) HS256, 8 h

      <-- {user, token}
      localStorage.ifqm_token = token
      localStorage.ifqm_org   = org_slug
```

One-time code

```
POST /api/auth/otp/request {identifier, purpose}
  resend throttle --> login_otps (latest row for identifier)
  resolveTenantByLogin(identifier) --> login_directory (unknown: generic reply, nothin
  SELECT users ... --> ifqm_<slug>.users
  INSERT login_otps {code_hash, tenant_id, tenant_slug, user_id, expires_at}
  smsService.sendSms and/or mailerService.send (Kaleyra / ZeptoMail)
POST /api/auth/otp/verify {identifier, code}
  SELECT login_otps ... consumed_at IS NULL AND expires_at > NOW()
  bcrypt.compare(code) ; attempts += 1 on miss
  UPDATE login_otps SET consumed_at = NOW() (single use, before minting)
  build the SAME session object, signToken(...), recordLogin
  <-- {user, token} browser: AuthContext.adoptSession(user, token, org_slug)
```

Every later request

```
Authorization: Bearer <jwt>
verifyToken (HS256, JWT_SECRET) expired -> 401 {expired:true}
payload.platform_admin ?
  yes: loadLivePlatformAdmin --> ifqm_master.platform_admins ; no req.db
  no : maintenanceStatus()
      attachTenantDb(payload.org_slug) --> tenants ; req.db = pool(ifqm_<slug>)
      loadLiveUser --> users row missing -> 401 ; status != active -> 401
      password_changed_at > token.pwd_ts -> 401 expire
      enforcePasswordChange, enforceBilling
  requireRole(...) compares req.user.role from the row, not the token
```

The important property for any external identity source: the token is only a pointer. Role, status and organisation are re-read from the database on every call, so revoking a person means updating their `users` row (set `status = 'inactive'` or bump `password_changed_at`); no token store needs to be searched.

11 Single sign-on integration

There is no SSO code in the repository today: sign-in is password or one-time code only. This section describes where an OpenID Connect or SAML integration fits the architecture above so that nothing downstream has to change. It is a design, not a description of existing code, and is written so the identity-provider side can be configured in parallel.

Where it plugs in

```
browser                kalpion-api                identity provider          database
-----
GET /login?org=vp
click "Sign in with
  <org> SSO"
GET /api/auth/sso/start?org=vp -->
    load tenant_sso for 'vp' -----> ifqm_ma
    build authorize URL (OIDC) or AuthnRequest (SAML)
    state = signed {org, nonce, return_to}

    <-- 302 to IdP
... user authenticates at the IdP (corporate credentials, MFA, etc.) ...
    <-- 302 /api/auth/sso/callback?code=...&state=... (OIDC)
    or POST /api/auth/sso/acs SAMLResponse=... (SAML)
--> callback / acs

    verify state ; exchange code / validate assertion
    claims: sub, email, name, groups?, employee_id?
    resolve tenant: from state.org, or by email domain
    find user: users WHERE sso_subject = sub OR email = claim.email -->
    not found and jit_provision = 1:
        userService.createUser(...) with random unusable hash
        directoryService.indexUser(tenant, user) -----> login_
        status != active: refuse
    update users.sso_subject, sso_last_login_at
    build the SAME session object ; signToken({user, org_slug, pwd_ts})
    recordLogin(actorType tenant_user) -----> platf
    <-- 302 https://kalpion.ifqm.org.in/sso/complete#token=...&org=vp
/sso/complete page reads the fragment, calls AuthContext.adoptSession(user, token, org)
everything after this is section 10, "Every later request", unchanged
```

Three properties make this a bolt-on rather than a rewrite. The session object and JWT are built the same way by both existing flows, so the SSO callback mints exactly the token the middleware expects. The SPA already has `adoptSession()` for a token that did not come from the password form. And per-request authorisation reads the `users` row, so an SSO-created user is governed by the same role, status and billing checks as everyone else.

Tenant resolution

An identity provider is an organisation-level thing, so the tenant must be known before the redirect to the IdP, not after. Two ways, in order of preference:

1. The organisation code, carried in the signed `state` (OIDC) or `RelayState` (SAML). The login page already collects it and stores it in `localStorage.ifqm_org`.
2. The email domain of the address the person types, matched against a new `tenant_sso.email_domains` list. `tenant_registrations.email_domain` is the natural seed for that list, but it is a registration-time fact and is not kept current, so it should be copied, not referenced.

IdP-initiated sign-in (the user starts at the IdP portal) only works with option 1 encoded in the configured `RelayState`.

Matching and provisioning users

Match on the IdP's stable subject first and `users.email` second, within the resolved tenant, then persist the subject so a later mailbox rename does not create a duplicate. Recommended attribute mapping, given the columns that exist:

IDP CLAIM / ATTRIBUTE	KALPION COLUMN	NOTES
sub / NameID	users.sso_subject (new)	Stable key; unique per tenant. Match on this first, email second.
email	users.email	Lower-cased; unique per tenant. Also indexed in login_directory.
name (or given + family)	users.name	Required, 100 chars.
employeeId / employeeNumber	users.employee_id	Required and unique per tenant. If the IdP cannot supply one, derive it from the subject, e.g. SSO- <hash>.
department	users.department	Optional.
manager (email or id)	users.manager_id	Optional; resolve to an existing user's id in the same tenant. This drives sequential approval routing, so an unmapped manager means ideas route to the committee or stall. Consider a sync rather than JIT for this field.
groups / roles	users.role	Map IdP group names to the eleven- value enum through tenant_sso.role_map; default employee. Never map to super_admin from a claim without an allow-list.
—	users.password_hash	NOT NULL; write a random bcrypt hash nobody knows. Leave must_change_password = 0 or the middleware will trap the person on the password page.
—	users.phone	Not needed for SSO, but without it the person cannot use SMS one- time codes as a fallback.

Configuration storage

IdP settings are per organisation and are read before a tenant pool is opened, so they belong in the registry rather than in `org_settings`. One new master table, created by a `_master.sql` migration:

```
CREATE TABLE tenant_sso (  
  tenant_id      INT PRIMARY KEY,           -- logical link to tenants.id  
  enabled        TINYINT(1) NOT NULL DEFAULT 0,  
  enforced       TINYINT(1) NOT NULL DEFAULT 0, -- 1: password / OTP sign-in refused  
  protocol       ENUM('oidc','saml') NOT NULL,  
  issuer         VARCHAR(255) NOT NULL,      -- OIDC issuer URL or SAML entityID  
  metadata_url   VARCHAR(500) NULL,         -- discovery document or SAML metadata  
  client_id      VARCHAR(255) NULL,         -- OIDC  
  client_secret_enc VARBINARY(1024) NULL,    -- OIDC, encrypted with a key from .env  
  idp_cert_pem   TEXT NULL,                 -- SAML signing certificate  
  email_domains  VARCHAR(500) NULL,         -- comma list, for domain-based resolution  
  jit_provision  TINYINT(1) NOT NULL DEFAULT 1,  
  default_role   VARCHAR(30) NOT NULL DEFAULT 'employee',  
  role_map       TEXT NULL,                 -- JSON: {"IdP group": "kalpion role"}  
  attr_map       TEXT NULL,                 -- JSON: claim name overrides  
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

And two columns on the tenant `users` table through an ordinary migration: `sso_subject` `VARCHAR(255) NULL UNIQUE` and `sso_last_login_at` `DATETIME NULL`. Nothing else in the schema changes.

Values the identity provider will ask for

SETTING	VALUE
Service-provider entity id / client name	https://kalpion.ifqm.org.in
OIDC redirect URI	https://kalpion.ifqm.org.in/api/auth/sso/callback
SAML assertion consumer service (ACS)	https://kalpion.ifqm.org.in/api/auth/sso/acs (HTTP-POST binding)
SAML SP metadata	https://kalpion.ifqm.org.in/api/auth/sso/metadata org=<slug>
Sign-out redirect	https://kalpion.ifqm.org.in/login
Scopes (OIDC)	openid email profile
NameID format (SAML)	persistent, or emailAddress if persistent is unavailable
Claims to release	email, name, employee id, department, groups (if roles are to be mapped)



Endpoints and code to add

PIECE	LOCATION	ROLE
GET /api/auth/sso/start	routes/authRoutes.js , behind authLimiter	Resolve tenant redirect
GET /api/auth/sso/callback , POST /api/auth/sso/acs	same router	Validate token
GET /api/auth/sso/metadata	same router, public	SAML
services/ssoService.js	new	Protocol as op saml/ mapp user dire then t from
Session builder	authService.js , otpService.js	Extract sign repea expor flows
Enforcement	authService.login , otpService.requestOtp	When 1 , ref to SSC supe
Admin UI	PlatformTenantsPage (or AdminPage)	Who r policy the sa lives i
/sso/complete	frontend/src/App.jsx , public route	Read t fragm clear t /das

Constraints to respect

- The API sits behind a proxy with `FORCE_HTTPS` ; callback URLs must be registered as `https` , and the redirect back to the SPA must carry the token in a URL fragment, never a query string, which the proxy log and browser history would keep.
- Put the start and callback routes behind `authLimiter` like every other sign-in endpoint.
- Maintenance mode and billing holds are enforced in the middleware after sign-in; the SSO callback should call `assertNotInMaintenance()` and check `heldForNonPayment` before minting, as password login does.
- Platform administrators are a different table with a mandatory second channel; keep them on the existing flow. SSO is for tenant users.
- Deactivation in the IdP does not reach Kalpion by itself. Either enforce SSO, so a person the IdP refuses cannot sign in at all, or add a SCIM endpoint or scheduled sync that sets `users.status = 'inactive'` ; the hourly housekeeping job in the scheduler is the natural place for a pull-based sync.
- The reporting line (`manager_id`) is the routing backbone. If the IdP holds manager data, sync it; if not, admins keep maintaining it in Kalpion as they do today.

12 A request, end to end

Submitting an idea, as the pieces above see it:

```

1  SubmitPage.jsx builds the multi-step form; on submit calls
   api.post('/ideas', body)                                services/api.js adds Bearer JWT
2  proxy -> kalpion-api :4000, x-forwarded-proto: https
3  app.js pipeline: helmet, https check, CORS, json, rate limit
4  routes/ideaRoutes.js  POST /    [requireAuth, tenantQuota]
5  middleware/auth.js    verify JWT -> attachTenantDb(org_slug)
                           req.db = pool for ifqm_vp; loadLiveUser re-reads the row
6  ideaController.create validates shape, calls ideaService.create(req.db, user, body)
7  ideaService            INSERT ideas; duplicate check; co-suggesters; attachments;
                           aiService.score (OpenAI/Gemini or heuristic) -> ai_score;
                           approvalStages picks first reviewer from manager chain;
                           idea_workflow row; notifications row; email_queue row;
                           users.points += 10
8  respond(res, {success:true, idea})                      JSON back to the browser
9  later, scheduler drains email_queue through mailerService -> ZeptoMail

```

13 Background work

There is no separate worker process. `server.js` starts `workers/scheduler.js` in the API container, which uses plain `setInterval` timers with an overlap guard:

JOB	CADENCE	WHAT IT DOES
Email queue	every 60 s (<code>EMAIL_QUEUE_INTERVAL_MS</code>)	For each tenant, sends pending rows in <code>email_queue</code> , records attempts, retires the stale backlog
Housekeeping	hourly	Prunes expired OTPs, purges logs past retention, re-checks subscription and SLA states, retries unsent registration notices
Backup	every <code>BACKUP_EVERY_HOURS</code> , if set	Runs <code>scripts/backup.js</code> , dumping every <code>ifqm_%</code> schema and keeping the last N runs

Because the timers live in the API process, running more than one API replica would run every job more than once; the design assumes a single instance.

14 External services

SERVICE	USED FOR	NOTES
ZeptoMail	All email	<code>PLATFORM_MAIL_TRANSPORT</code> = smtp, api or auto; auto tries SMTP on 465 then falls back to the HTTPS API. One platform sender; tenants do not bring their own mail.
Kaleyra	SMS one-time codes	Indian DLT: sender id and per-purpose template ids are registered content; the API refuses to start a template that is not configured. Deliveries are logged in <code>sms_delivery_log</code> .
OpenAI or Gemini	Idea scoring	Optional. <code>AI_PROVIDER</code> unset selects a built-in heuristic scorer so the product works with no key.
Razorpay	Subscription payments	Plans in <code>ifqm_master.plans</code> ; attempts and events recorded per tenant; a grace period before a non-paying tenant is held.
QCMS	Push of approved ideas	Each tenant stores its own API key; usage is metered in <code>tenant_api_usage</code> .

15 Build, test and deployment

CI (`ci.yml`) runs on every push to `main` and every pull request: the backend invariant suite against MariaDB 10.11 on Node 22, then a frontend build. The suite provisions its own `ifqm_test_*` schemas and forces `STRICT_ALL_TABLES`, so local, CI and production agree on what the database accepts.

Deploy (`deploy.yml`) runs only after CI succeeds on `main`:

```
1 tar the tested commit (excluding .git, node_modules, backend/.env, uploads, backups)
2 scp to the VPS, ssh in, cd /opt/ifqm-kalpion
3 save backend/.env, untar over the tree, restore backend/.env
4 docker compose build --pull kalpion-api kalpion-web
5 docker compose up -d --no-deps kalpion-api kalpion-web      (db is never rebuilt)
6 wait for MariaDB; docker exec kalpion-api npm run migrate
7 poll /api/health inside the container with x-forwarded-proto: https
  until it answers 200 {"status":"ok"}; fail the deploy if it never does
8 wget --spider https://kalpion.ifqm.org.in
```

The API image is `node:22-alpine` with production dependencies and `backend/` copied in; `.dockerignore` keeps `.env`, uploads and backups out of the image. The web image builds the SPA with `VITE_API_URL=https://kalpion.ifqm.org.in/api` and serves it from nginx. Uploads persist through a bind mount (`./backend/uploads`); the database through a named volume. The database container reads its root and application passwords from a compose-level `.env` only on first initialisation.

Earlier hosting (Render for the API, Vercel for the SPA, Aiven for MySQL 8) is retained in `render.yaml` and `vercel.json` but is no longer the production target.

16 Configuration

The backend reads `backend/.env` and nothing else; `backend/.env.example` is the committed template. Per-deployment values are kept outside git as `.env.<half>.<target>` at the repository root and copied into place at deploy time. On start-up `assertConfigOrExit` refuses to run with an empty database password or a missing JWT secret, and logs a warning for each SMS template that is not configured.

GROUP	KEYS
-------	------

Server	<code>PORT</code> , <code>NODE_ENV</code> , <code>FORCE_HTTPS</code> , <code>TRUST_PROXY</code> , <code>CORS_ORIGIN</code> , <code>FRONTEND_BASE_URL</code> , <code>JWT_SECRET</code>
--------	--

Database	<code>MASTER_DB_HOST/USER/PASS/NAME</code> , <code>APP_DB_USER/PASS</code> , <code>DB_PORT</code> , <code>DB_SSL</code> , <code>DB_SSL_CA</code> , <code>DB_POOL_SIZE</code> , <code>DB_MAX_POOLS</code>
----------	---

Mail	<code>PLATFORM_SMTP_HOST/PORT/USER/PASS</code> , <code>PLATFORM_MAIL_API_KEY</code> , <code>PLATFORM_MAIL_TRANSPORT</code> , <code>PLATFORM_MAIL_FROM</code> , <code>EMAIL_QUEUE_INTERVAL_MS</code>
------	--

SMS	<code>SMS_PROVIDER</code> , <code>SMS_ENDPOINT</code> , <code>SMS_SID</code> , <code>SMS_API_KEY</code> , <code>SMS_SENDER_ID</code> , <code>SMS_PE_ID</code> , per-purpose template ids, <code>OTP_ENABLED</code>
-----	---

Optional	<code>AI_PROVIDER</code> , <code>OPENAI_API_KEY</code> , <code>GEMINI_API_KEY</code> , Razorpay keys, <code>QCMS_BASE_URL</code> , <code>BACKUP_EVERY_HOURS</code> , <code>GLOBAL_RATE_LIMIT</code> , <code>AUTH_RATE_LIMIT</code>
----------	---

Derived from the repository as of commit 0761264. File and route names are the actual ones; counts (pages, tables, migrations) are as of that commit.